



Week 2: Arrays



Dr. Sirasit Lochanachit



Today's Outline

1. What is an Array?
2. Static vs Dynamic Array
3. Array Operations and Asymptotic Performance



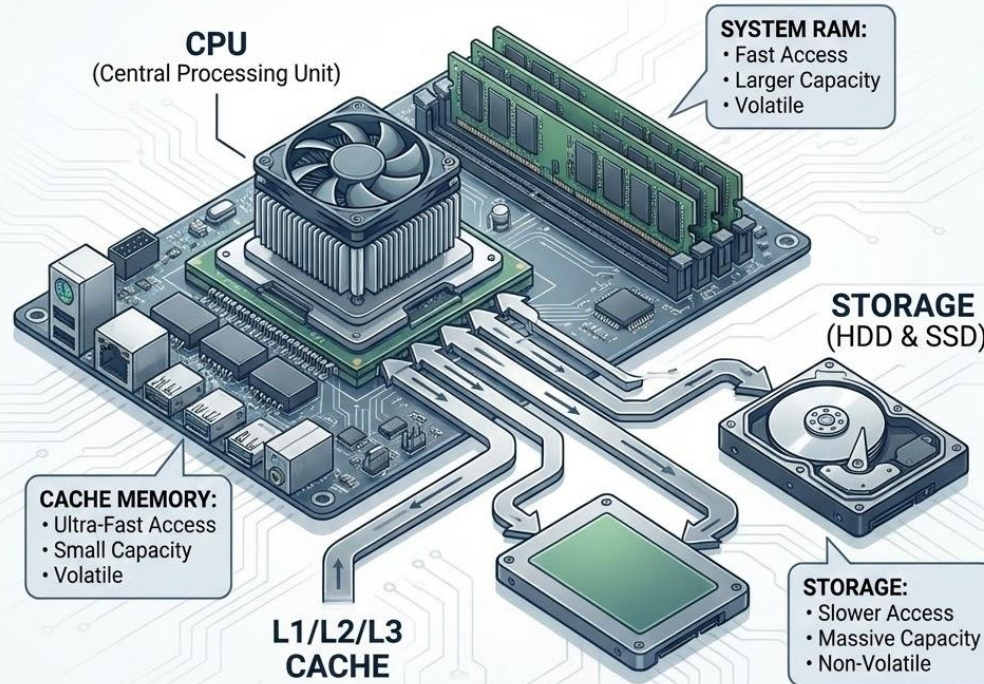
Python Sequence Types



- Built-in class
 - list
 - tuple
 - str

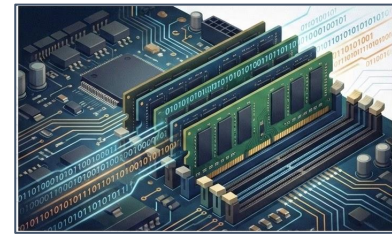
What are the characteristics shared amongst these 3 classes?

COMPUTER MEMORY: COMPONENTS & HIERARCHY





Computer Memory



- A computer will have a large number of bytes of memory.

Computer Memory



Memory Address Tracking:
Stores data with a unique location for easy retrieval.

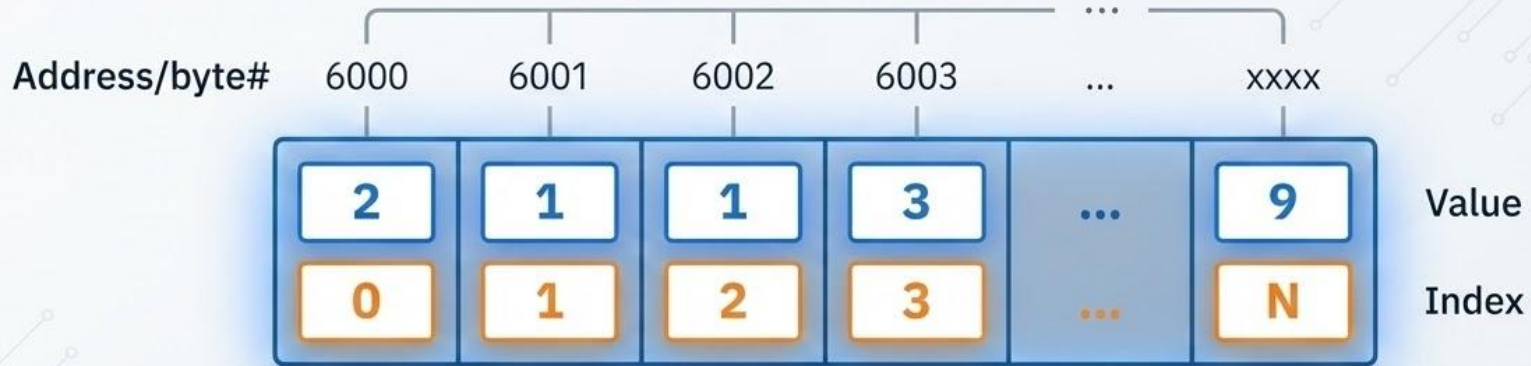


Unique Byte Addressing:
Each byte is assigned a distinct, sequential numerical address.



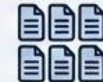
Constant Time Access:
Any byte can be read or written instantly, regardless of its position, with $O(1)$ time complexity.

What is an Array?



An array is a **chunk of memory**, consisting of **equal-size** elements.

$\begin{matrix} 1 \\ 2 \\ 3 \end{matrix}$ — Each of those elements has an **integer index**, which uniquely refers to the value stored.



The values are all of the **same type** (integer, character, etc.).

Array of Characters



In Python, it represents a **Unicode character** with 16 bits (i.e., 2 bytes).



Since each cell has an equal-size, any element can be accessed constantly with a formula:
 $\text{start_address} + (\text{cell_size} * \text{index})$



Exercise: How computer can access any element

Given an array:



Start Address: 6000

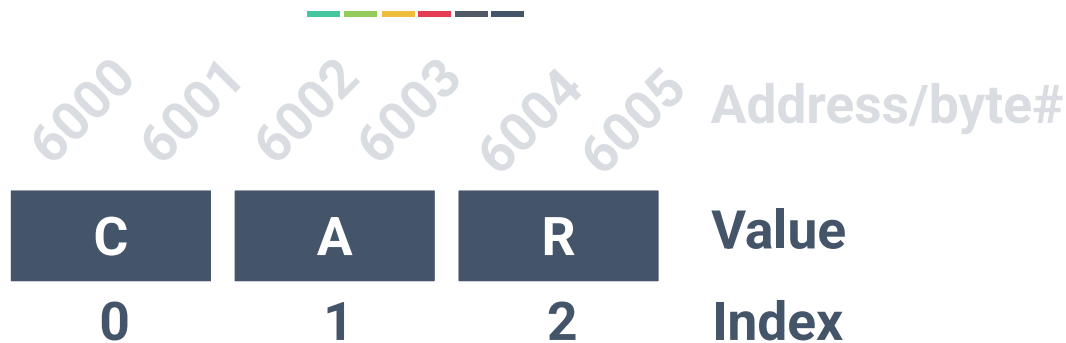


Cell Size: 8

What is the address of the element at index 6?



Array of Characters



- Luckily, a programming language calculate memory addresses of an array automatically.

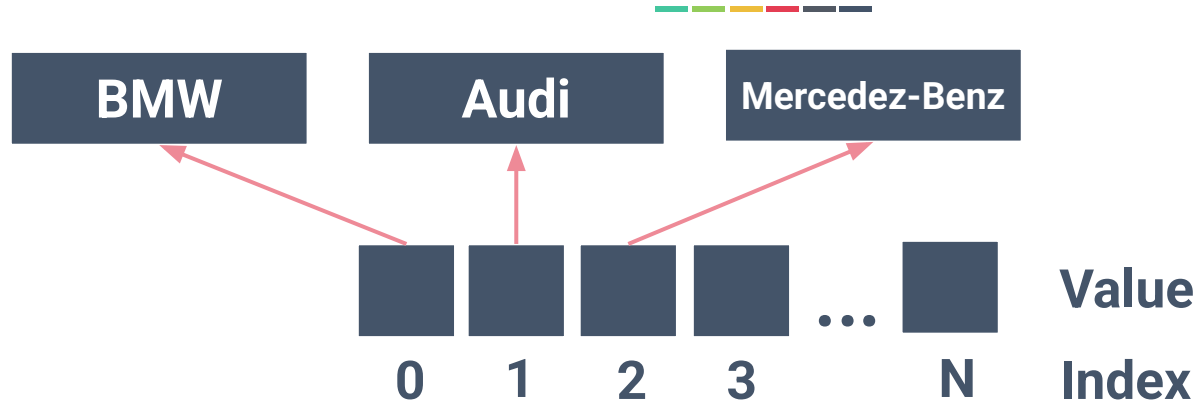


Array of Strings?



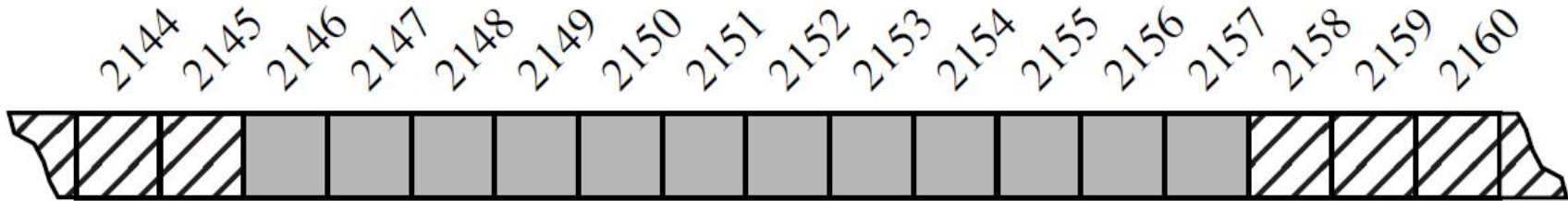
- Can we use an array to store strings of characters?

Python List



- Python list is a referential-type array that stores the memory addresses (references) of a value instead of the value itself.
- Strings can be in any length, but memory addresses are fixed-size.

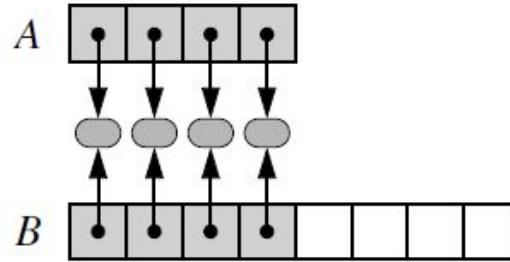
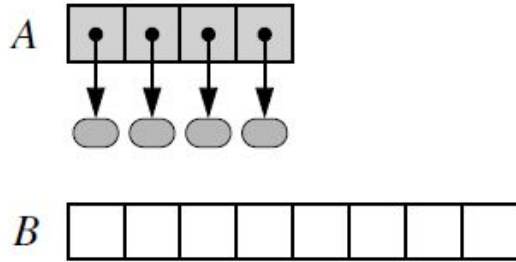
Static Array



- Once an array in C or Java has been created, it's size is fixed.
- Python tuple and str instances are immutable.
 - Unable to change size and value

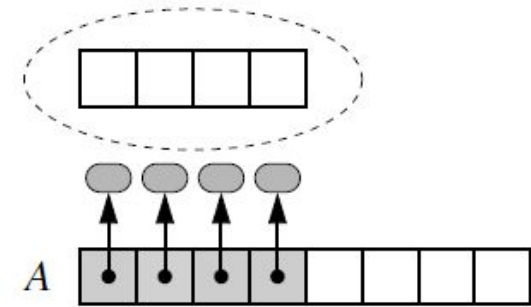
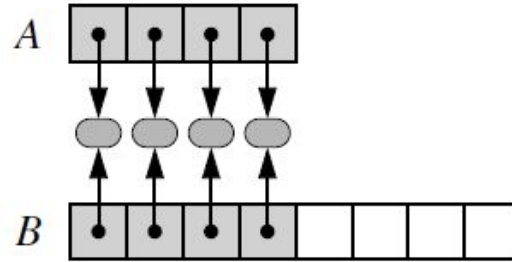
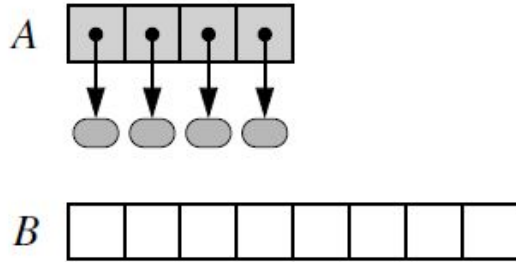


Dynamic Array



- Python's list instance maintains an underlying array that often has greater capacity than the current length of the list.
- If a capacity is exhausted, the list class requests a new, larger array.

Dynamic Array



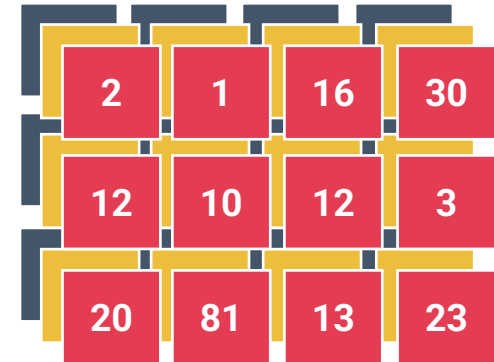
Arrays



(a) One dimension

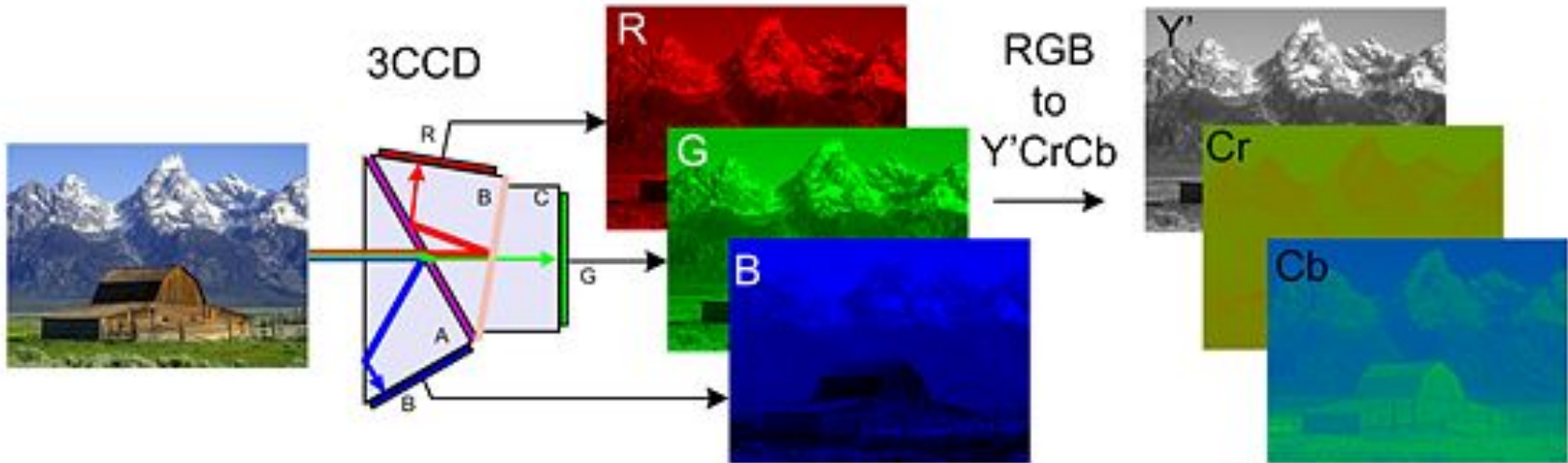


(b) Two dimensions



(c) Three dimensions

3D Arrays





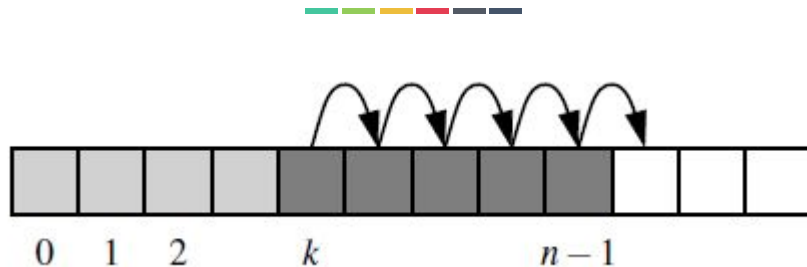
Python-based List Operations



Operation	Running Time
<code>len(data)</code>	$O(1)$
<code>data[i]</code>	$O(1)$
<code>data[i] = val</code>	$O(1)$
<code>c * data</code>	$O(n)$
<code>data.reverse()</code>	$O(n)$
<code>data.sort()</code>	$O(n \log n)$



Adding Elements to a List



- Operation **insert**(k , *value*) requires creating a room to insert a new element at index k of a dynamic array
 - Make room by shifting forward the $(n - 1) - k$ elements $data[k], \dots, data[n-1]$
 - $O(n - k + 1)$ for inserting at index k



Data

10	5	8				
----	---	---	--	--	--	--

Best case (location) for inserting a value?

Worst case (location) for inserting a value?



Lab 3-1



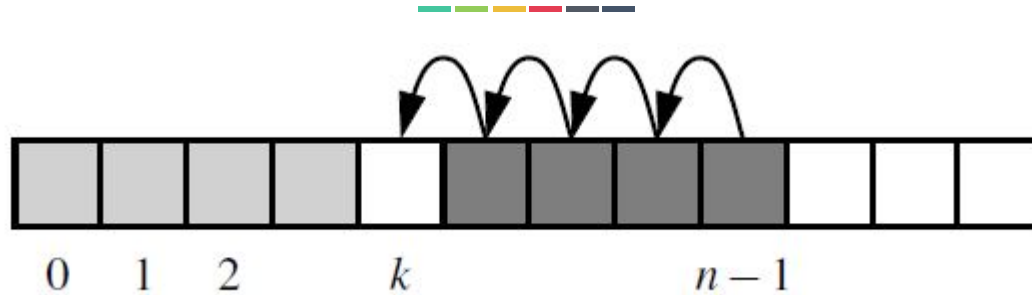
Record the average running time of `insert(k, 20)` in seconds with three different inserting patterns for each of the N calls:

1. Repeatedly insert at the beginning of a list
 2. Repeatedly insert near the middle of a list
 3. Repeatedly insert at the end of the list
- Each pattern starts from an empty list `[]` -> `data = []`

Lab 3-1

	N			
Patterns	10	20	50	100
First Case				
Second Case				
Third Case				

Removing Elements from a List



- Operation **pop**(k) removes the element that is at index $k < n$ of a list
 - Then shifting backward the $(n - 1) - (k + 1)$ elements $data[k+1], \dots, data[n-1]$
 - $O(n - k)$ for removing element at index k



Data

10	5	8				
----	---	---	--	--	--	--

Best case (location) for removing a value?

Worst case (location) for removing a value?

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

10	5	8	7	1			
----	---	---	---	---	--	--	--

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

--	--	--	--	--	--	--	--

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

--	--	--	--	--	--	--	--

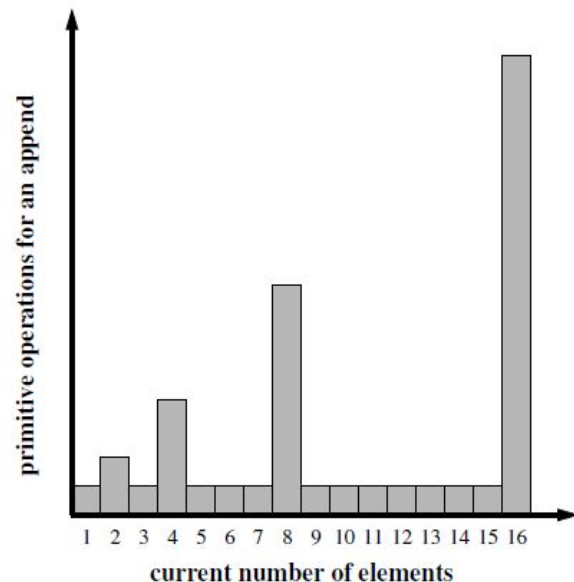


Amortization (Distribution) Analysis

- Operation **append**(*value*)
 - If an array is not full, copy the value into the array - $O(1)$
 - Otherwise, grow a dynamic array by
copy the old values to a new array - $O(n)$

Does this mean **append**() has $O(n)$?

- Find out in Lab 4-3





Lab 4-3



- Operation **append**(*value*)
 - If an array is not full, copy the value into the array - $O(1)$
 - Otherwise, grow a dynamic array by copy the old values to a new array - $O(n)$

Growing array size by 1 or double?

Which option is more efficient in amortized time?